

Shadowing is good for safety

Document number	P2951R0
Date	2023-7-14
Reply-to	Jarrad J. Waterloo <descender76 at gmail dot com>
Audience	SG23 Safety and Security Evolution Working Group (EWG) Library Evolution Working Group (LEWG)

Shadowing is good for safety

Table of contents

- Shadowing is good for safety
 - Abstract
 - Motivational Examples
 - Summary
 - Frequently Asked Questions
 - References

Abstract

Removing arbitrary limitations in the language associated with shadowing can improve how programmers deal with invalidation safety issues. It can also benefit a programmer's use of const correctness which impacts immutability and thread safety concerns. Regardless, it promotes simple and succinct code.

Motivational Examples

1st request: It would be beneficial if programmers could shadow a variable with void initialization instead of having to resort to a tag class.

removing names

Present Workaround	Request
<pre>#include <string> #include <vector> using namespace std; struct dename{}; // alt: unname, useless int main() { vector<string> vs{"1", "2", "3"}; for (auto &s : vs) { dename vs; // this helps prevents iterator // and reference invalidation // as the programmer can't use // the container being worked on } }</pre>	<pre>#include <string> #include <vector> using namespace std; int main() { vector<string> vs{"1", "2", "3"} for (auto &s : vs) { void vs; // this helps prevents iterator // and reference invalidation // as the programmer can't use // the container being worked } }</pre>

Even if this feature could not be standardized as a language feature, by removing a non breaking restriction, the tag class is adequate, provided the tag class name could be standardized as a library feature.

2nd request: It would be beneficial if programmers could initialize shadowed variables with the variable that is being shadowed.

shadowing limitation: initialization

Present and Request	Present Workaround
<pre>#include <string> #include <vector> using namespace std; int main() { vector<string> vs{"1", "2", "3"}; for (auto &s : vs) { // allow programmer to only call // non container changing methods const vector<string>& vs = vs;// error } return 0; }</pre>	<pre>#include <string> #include <vector> using namespace std; struct dename{}; int main() { vector<string> vs{"1", "2", "3"}; for (auto &s : vs) { // allow programmer to only // non container changing m const vector<string>& vs1 = dename vs; // superfluous naming // superfluous line of code } return 0; }</pre>

Implementation Experience

gcc	this works in gcc
clang	warning: reference 'vs' is not yet bound to a value when used within its own initialization [-Wuninitialized]
msvc	warning C4700: uninitialized local variable 'vs' used

3rd request: It would be beneficial if programmers could shadow variables without having involve a child scope.

shadowing limitation: same level

Present and Request	Present Workaround
<pre data-bbox="133 569 873 1140">#include <string> #include <vector> using namespace std; int main() { vector<string> vs{"1", "2", "3"}; // done doing complex initializaton // want it immutable here on out const vector<string>& vs = vs;// error return 0; }</pre>	<pre data-bbox="928 275 1521 1434">#include <string> #include <vector> using namespace std; struct dename{}; int main() { vector<string> vs{"1", "2", "3"} // done doing complex initializ // want it immutable here on ou { const vector<string>& vs1 = v dename vs; // superfluous naming // superfluous lines of code // superfluous level } // OR use immediately invoked // lambda function [](const vector<string>& vs) { // superfluous lines of code // superfluous level }(vs); return 0; }</pre>

The error in the Present and Request example may read something like “error: conflicting declaration ‘const vector<string> vs’//note: previous declaration as ‘vector<string> vs’”.

Summary

The advantages of adopting said proposal are as follows:

1. It better allows programmers to use the compiler to avoid, debug and identify iterator, reference and pointer invalidation issues.

2. This proposal aids in const correctness which is good for immutability and thread safety.
3. More shadowing promotes simple and succinct code.

Frequently Asked Questions

Why `dename` instead of `unname`?

1. `dename` ^[1]: To remove the name from
2. `unname` ^[2]: To cease to name; to deprive (someone or something) of their name

While both names would work, `dename` seems simpler to me than `unname`. Personally, I think using `void` would be the more intuitive C++ name.

References

1. <https://en.wiktionary.org/wiki/dename> (<https://en.wiktionary.org/wiki/dename>) ↩
2. <https://en.wiktionary.org/wiki/unname> (<https://en.wiktionary.org/wiki/unname>) ↩